СТРУКТУРА ПРОЕКТА

```
├── README.md
├── effects.js
├── favicon.png
├── index.html
├── loader.js
├── script.js
└── style.css
```

ИСХОДНЫЙ КОД ФАЙЛОВ

=== ФАЙЛ: README.md ===

```
# 🌌 AstroTrails

> Автономный веб-редактор для создания фотографий звездных треков в режиме «Кометы» (Comet Mode) прямо из ваших таймлапс-видео.

### 🚀 [Запустить AstroTrails в браузере (GitHub Pages)](https://na1x-dev.github.io/AstroTrails/)

---

## 🌟 Основные особенности
- **Режим кометы нового поколения:** Использование нелинейных геометрических кривых для бесконечно длинного и плавного затухания хвостов звезд.
- **Экспорт в Оригинальном Качестве:** Рендеринг итогового изображения выполняется в 100% разрешении исходного видео (поддержка 4K UHD).
- **GPU-Оптимизация (Proxy-Preview):** Мгновенная цветокоррекция на 60 FPS за счет интеллектуального масштабирования превью-потока.
- **Интерактивное управление:** Регулировка ползунков с помощью кнопок точного шага `+` / `-` и колесика мыши.
- **Абсолютная автономность:** Работает полностью в браузере (Client-Side) без серверов, баз данных и внешних зависимостей.
- **Mobile Responsive:** Полная адаптация интерфейса для удобного использования на экранах смартфонов и планшетов.

## 🎛 Панель инструментов постобработки
- **Трассы:** Настройка крутизны экспоненциального затухания хвоста кометы.
- **Экспозиция:** Тонкая регулировка яркости и контрастности кадра.
- **Гамма:** Коррекция полутонов для создания глубокого, абсолютно черного ночного неба.
- **RGB микшер:** Раздельное управление Красным, Зеленым и Синим каналами для проявления туманностей и млечного пути.

## 🛠 Быстрый старт на локальном ПК
1. Склонируйте репозиторий:
   ```bash
   git clone https://github.com/Na1x-dev/AstroTrails.git
   ```
2. Откройте файл `index.html` в любом современном браузере.
3. Перетащите ваше склеенное видео астрофотографии в центр экрана!
```

=== ФАЙЛ: effects.js ===

```
const effects = {
    brightness: document.getElementById('brightness'),
    contrast: document.getElementById('contrast'),
    gamma: document.getElementById('gamma'),
    channelR: document.getElementById('channelR'),
    channelG: document.getElementById('channelG'),
    channelB: document.getElementById('channelB')
};

// Привязываем события и обновляем проценты по корректным ID
Object.keys(effects).forEach(key => {
    if (window.enhanceSlider) {
        window.enhanceSlider(effects[key]);
    }

    effects[key].addEventListener('input', () => {
        let suffix = '%'; if(key === 'gamma') suffix = '';
```

```
        const displayVal = key === 'gamma' ? (effects[key].value / 100).toFixed(1) : effects[key].value;

        // КЛЮЧЕВОЙ ФИКС: Точное совпадение ID с HTML-структурой
        const valElement = document.getElementById(`${key}Val`);
        if (valElement) {
            valElement.textContent = displayVal + suffix;
        }

        // Мгновенно применяем стили
        window.applyPostProcessing();
    });
});

window.resetAllEffects = function() {
    Object.keys(effects).forEach(key => {
        effects[key].value = 100;
        let suffix = '%'; if(key === 'gamma') suffix = '';
        const displayVal = key === 'gamma' ? '1.0' : '100';

        const valElement = document.getElementById(`${key}Val`);
        if (valElement) {
            valElement.textContent = displayVal + suffix;
        }
    });

    // Принудительно вызываем обновление холста
    if (window.applyPostProcessing) {
        window.applyPostProcessing();
    }
};

window.toggleFiltersState = function(isDisabled) {
    Object.keys(effects).forEach(key => {
        effects[key].disabled = isDisabled;
        const steps = effects[key].parentNode.querySelectorAll('.btn-step');
        steps.forEach(b => { b.disabled = isDisabled; });
    });
};

window.applyPostProcessing = function() {
    if (!window.astroState || !window.astroState.previewBuffer) return;

    const canvas = document.getElementById('glCanvas');
    const ctx = canvas.getContext('2d');

    const imgData = new ImageData(
        new Uint8ClampedArray(window.astroState.previewBuffer.data),
        window.astroState.previewBuffer.width,
        window.astroState.previewBuffer.height
    );
    const data = imgData.data;

    const b = parseInt(effects.brightness.value) / 100;
    const c = parseInt(effects.contrast.value) - 100;
    const g = parseInt(effects.gamma.value) / 100;
    const rM = parseInt(effects.channelR.value) / 100;
    const gM = parseInt(effects.channelG.value) / 100;
    const bM = parseInt(effects.channelB.value) / 100;

    const factor = (259 * (c + 255)) / (255 * (259 - c));

    for (let i = 0; i < data.length; i += 4) {
        let r = data[i] * b * rM;
        let gChan = data[i+1] * b * gM;
        let bl = data[i+2] * b * bM;

        if (c !== 0) {
            r = factor * (r - 128) + 128;
            gChan = factor * (gChan - 128) + 128;
            bl = factor * (bl - 128) + 128;
        }

        if (g !== 1) {
            r = 255 * Math.pow(Math.clamp(r, 0, 255) / 255, 1 / g);
```

```
            gChan = 255 * Math.pow(Math.clamp(gChan, 0, 255) / 255, 1 / g);
            bl = 255 * Math.pow(Math.clamp(bl, 0, 255) / 255, 1 / g);
        }

        data[i] = r < 0 ? 0 : (r > 255 ? 255 : r);
        data[i+1] = gChan < 0 ? 0 : (gChan > 255 ? 255 : gChan);
        data[i+2] = bl < 0 ? 0 : (bl > 255 ? 255 : bl);
    }

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const tempCanvas = document.createElement('canvas');
    tempCanvas.width = imgData.width;
    tempCanvas.height = imgData.height;
    tempCanvas.getContext('2d').putImageData(imgData, 0, 0);

    ctx.drawImage(tempCanvas, 0, 0, canvas.width, canvas.height);
};

Math.clamp = function(value, min, max) {
    return Math.min(max, Math.max(min, value));
};

document.getElementById('downloadBtn').addEventListener('click', (e) => {
    e.stopImmediatePropagation();
    if (!window.astroState || !window.astroState.rawCompositeBuffer) return;

    const originalBuffer = window.astroState.rawCompositeBuffer;
    const exportCanvas = document.createElement('canvas');
    exportCanvas.width = originalBuffer.width;
    exportCanvas.height = originalBuffer.height;
    const exportCtx = exportCanvas.getContext('2d');

    const imgData = new ImageData(
        new Uint8ClampedArray(originalBuffer.data),
        originalBuffer.width,
        originalBuffer.height
    );
    const data = imgData.data;

    const b = parseInt(effects.brightness.value) / 100;
    const c = parseInt(effects.contrast.value) - 100;
    const g = parseInt(effects.gamma.value) / 100;
    const rM = parseInt(effects.channelR.value) / 100;
    const gM = parseInt(effects.channelG.value) / 100;
    const bM = parseInt(effects.channelB.value) / 100;
    const factor = (259 * (c + 255)) / (255 * (259 - c));

    for (let i = 0; i < data.length; i += 4) {
        let r = data[i] * b * rM;
        let gChan = data[i+1] * b * gM;
        let bl = data[i+2] * b * bM;

        if (c !== 0) {
            r = factor * (r - 128) + 128;
            gChan = factor * (gChan - 128) + 128;
            bl = factor * (bl - 128) + 128;
        }

        if (g !== 1) {
            r = 255 * Math.pow(Math.clamp(r, 0, 255) / 255, 1 / g);
            gChan = 255 * Math.pow(Math.clamp(gChan, 0, 255) / 255, 1 / g);
            bl = 255 * Math.pow(Math.clamp(bl, 0, 255) / 255, 1 / g);
        }

        data[i] = r < 0 ? 0 : (r > 255 ? 255 : r);
        data[i+1] = gChan < 0 ? 0 : (gChan > 255 ? 255 : gChan);
        data[i+2] = bl < 0 ? 0 : (bl > 255 ? 255 : bl);
    }

    exportCtx.putImageData(imgData, 0, 0);

    const link = document.createElement('a');
    link.download = `AstroTrails_HD_${Date.now()}.png`;
```

```
    link.href = exportCanvas.toDataURL('image/png');
    link.click();
});

window.toggleFiltersState(true);
```

=== ФАЙЛ: index.html ===

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
    <title>AstroTrails</title>
    <link rel="preconnect" href="https://googleapis.com">
    <link rel="preconnect" href="https://gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cloudflare.com">
    <link rel="stylesheet" href="style.css">
    <link rel="icon" href="favicon.png" type="image/x-icon">

</head>
<body>

<div class="editor-layout">
    <!-- Левая панель инструментов -->
    <aside class="sidebar">
        <div class="sidebar-header">
            <img src="favicon.png" alt="Logo" class="app-logo-img" onerror="this.style.display='none';
document.getElementById('fallbackLogo').style.display='block';">
            <i class="fa-solid fa-meteor logo-icon" id="fallbackLogo" style="display:none;"></i>
            <div>
                <h2>AstroTrails</h2>
            </div>
        </div>

        <div class="sidebar-content">
            <!-- Краткий туториал -->
            <div class="tutorial-card" id="tutorialBlock">
                <div class="tutorial-header">
                    <span><i class="fa-solid fa-circle-info"></i> Как пользоваться?</span>
                    <button id="closeTutorialBtn"><i class="fa-solid fa-xmark"></i></button>
                </div>
                <ol class="tutorial-steps">
                    <li>Загрузите **одно или несколько** видео-таймлапсов звездного неба.</li>
                    <li>Нажмите **«Сгенерировать треки»**.</li>
                    <li>Используйте ползунки **Постобработки**, чтобы сделать небо темнее, а звёзды ярче.</li>
                </ol>
            </div>

            <!-- Секция 1: Управление файлами -->
            <div class="section-card">
                <div class="section-title">Импорт медиа</div>
                <button class="btn btn-tonal" id="uploadBtnClick">
                    <i class="fa-solid fa-images"></i>
                    Выбрать видеоролики...
                </button>
                <!-- Добавлен атрибут multiple -->
                <input type="file" id="videoInput" accept="video/*" multiple style="display: none;">
                <div id="fileInfoText">Файлы не выбраны</div>
            </div>

            <!-- Секция 2: Алгоритм треков -->
            <div class="section-card disabled-control" id="secTracks">
                <div class="section-title">Настройки трасс</div>
                <div class="slider-group">
                    <div class="slider-header">
                        <span>Прозрачность хвоста</span>
                        <span class="val" id="opacityVal">20%</span>
                    </div>
                    <input type="range" id="startOpacity" min="1" max="99" value="20" disabled>
                </div>
                <button class="btn btn-primary" id="processBtn" disabled>
                    <i class="fa-solid fa-wand-magic-sparkles"></i>
```

```
            Сгенерировать треки
          </button>
        </div>

        <!-- Секция 3: Цветокоррекция -->
        <div class="section-card disabled-control" id="secEffects">
          <div class="section-title">Постобработка кадра (GPU)</div>

          <div class="slider-group">
            <div class="slider-header"><span>Яркость</span><span class="val" id="brightnessVal">100%</span></div>
            <input type="range" id="brightness" min="0" max="200" value="100" disabled>
          </div>

          <div class="slider-group">
            <div class="slider-header"><span>Контрастность</span><span class="val" id="contrastVal">100%</span></div>
            <input type="range" id="contrast" min="0" max="200" value="100" disabled>
          </div>

          <div class="slider-group">
            <div class="slider-header"><span>Гамма</span><span class="val" id="gammaVal">1.0</span></div>
            <input type="range" id="gamma" min="20" max="250" value="100" disabled>
          </div>

          <div class="divider"></div>

          <div class="slider-group rgb-slider r-slider">
            <div class="slider-header"><span>Красный канал (R)</span><span class="val" id="channelRVal">100%</span></div>
            <input type="range" id="channelR" min="0" max="200" value="100" disabled>
          </div>

          <div class="slider-group rgb-slider g-slider">
            <div class="slider-header"><span>Зеленый канал (G)</span><span class="val" id="channelGVal">100%</span></div>
            <input type="range" id="channelG" min="0" max="200" value="100" disabled>
          </div>

          <div class="slider-group rgb-slider b-slider">
            <div class="slider-header"><span>Синий канал (B)</span><span class="val" id="channelBVal">100%</span></div>
            <input type="range" id="channelB" min="0" max="200" value="100" disabled>
          </div>
        </div>
      </div>

      <div class="sidebar-footer">
        <button class="btn btn-tonal" id="resetBtn" disabled style="margin-bottom: 8px;">
          <i class="fa-solid fa-arrow-rotate-left"></i>
          Сбросить настройки
        </button>
        <button class="btn btn-tonal" id="downloadVideoBtn" disabled style="margin-bottom: 8px; display: none;">
          <i class="fa-solid fa-video"></i>
          Экспорт видео (MP4)
        </button>
        <button class="btn btn-primary" id="downloadBtn" disabled>
          <i class="fa-solid fa-download"></i>
          Экспорт в PNG
        </button>
      </div>
    </aside>

    <!-- Центральная область просмотра -->
    <main class="viewport" id="dropZone">
      <div class="canvas-container">
        <canvas id="outputCanvas" style="display:none;"></canvas>
        <canvas id="glCanvas"></canvas>

        <div class="viewport-overlay" id="statusOverlay">
          <div class="loader-spinner" id="loaderSpinner"></div>
          <i class="fa-solid fa-cloud-moon overlay-icon" id="statusIcon"></i>
          <div class="status-title" id="statusTitle">Пустой холст</div>
          <div class="status-desc" id="statusText">Перетащите пачку мини-таймлапсов прямо сюда в центр экрана.</div>

          <div class="progress-container" id="progressContainer">
            <div class="progress-bar" id="progressBar"></div>
          </div>
        </div>
```

```
        </div>
    </main>
</div>

<script src="script.js"></script>
<script src="effects.js"></script>
<script src="loader.js"></script>
</body>
</html>
```

=== ФАЙЛ: loader.js ===

```
// Пошаговое чтение пачки видеофайлов в режиме реального времени
document.getElementById('processBtn').addEventListener('click', async () => {
    if (!window.astroState.videoFiles || window.astroState.videoFiles.length === 0) return;

    if (window.toggleUIBlock) window.toggleUIBlock(true);

    const canvas = document.getElementById('glCanvas');

    // КРИТИЧЕСКИЙ ФИКС: Принудительно ПОКАЗЫВАЕМ оверлей при повторном старте
    const overlay = document.getElementById('statusOverlay');
    if (overlay) {
        overlay.classList.remove('hidden');
    }

    const oldIcon = document.getElementById('statusIcon');
    if (oldIcon) oldIcon.style.display = 'none';

    const spinner = document.getElementById('loaderSpinner');
    if (spinner) {
        spinner.className = 'google-spinner';
        spinner.style.display = 'block';
    }

    // Выставляем четкие стартовые надписи
    document.getElementById('statusTitle').textContent = 'Идет обработка видео';
    document.getElementById('statusText').textContent = 'Пожалуйста, подождите...';

    window.astroState.cachedFrames = [];
    const files = window.astroState.videoFiles;

    for (let fIdx = 0; fIdx < files.length; fIdx++) {
        if (files.length > 1) {
            document.getElementById('statusTitle').textContent = `Обработка видео: клип ${fIdx + 1} из ${files.length}`;
        }

        const video = document.createElement('video');
        const videoUrl = URL.createObjectURL(files[fIdx]);

        video.muted = true;
        video.playsInline = true;
        video.crossOrigin = "anonymous";
        video.src = videoUrl;

        await new Promise((resolve) => {
            video.oncanplaythrough = resolve;
            video.load();
        });

        if (fIdx === 0) {
            canvas.width = video.videoWidth || 1920;
            canvas.height = video.videoHeight || 1080;
        }

        const tempCanvas = document.createElement('canvas');
        tempCanvas.width = canvas.width;
        tempCanvas.height = canvas.height;
        const tempCtx = tempCanvas.getContext('2d');

        video.play();

        await new Promise((resolve) => {
            const trackInterval = setInterval(async () => {
```

```
            if (video.paused || video.ended) {
                clearInterval(trackInterval);
                resolve();
                return;
            }

            tempCtx.drawImage(video, 0, 0, canvas.width, canvas.height);

            const frameCopy = document.createElement('canvas');
            frameCopy.width = canvas.width;
            frameCopy.height = canvas.height;
            frameCopy.getContext('2d').drawImage(tempCanvas, 0, 0);

            window.astroState.cachedFrames.push(frameCopy);
        }, 40);

        video.onended = () => {
            clearInterval(trackInterval);
            resolve();
        };
    });

    URL.revokeObjectURL(videoUrl);
  }

  if (window.renderCompositeTracks) {
    window.renderCompositeTracks();
  }
});

// Алгоритм наложения сглаживания треков (Интеллектуальный Gap Filling)
window.renderCompositeTracks = function() {
  if (!window.astroState.cachedFrames || window.astroState.cachedFrames.length === 0) return;

  const canvas = document.getElementById('glCanvas');
  const ctx = canvas.getContext('2d');
  const startOpacityInput = document.getElementById('startOpacity');

  ctx.fillStyle = '#000000';
  ctx.fillRect(0, 0, canvas.width, canvas.height);

  const total = window.astroState.cachedFrames.length;
  const sliderVal = parseFloat(startOpacityInput.value);
  const powerCoefficient = Math.pow(1.06, sliderVal) - 0.9;

  for (let i = 0; i < total; i++) {
    const progress = i / (total - 1 || 1);
    const opacity = Math.pow(progress, powerCoefficient);

    ctx.save();
    ctx.globalAlpha = opacity;
    ctx.globalCompositeOperation = 'lighten';

    ctx.drawImage(window.astroState.cachedFrames[i], 0, 0);

    if (i > 0) {
      ctx.globalAlpha = opacity * 0.4;
      ctx.drawImage(window.astroState.cachedFrames[i], -0.5, -0.5);
    }
    ctx.restore();
  }

  window.astroState.rawCompositeBuffer = ctx.getImageData(0, 0, canvas.width, canvas.height);

  const previewWidth = Math.min(canvas.width, 1024);
  const previewHeight = (canvas.height / canvas.width) * previewWidth;
  const pCanvas = document.createElement('canvas');
  pCanvas.width = previewWidth;
  pCanvas.height = previewHeight;
  pCanvas.getContext('2d').drawImage(canvas, 0, 0, previewWidth, previewHeight);
  window.astroState.previewBuffer = pCanvas.getContext('2d').getImageData(0, 0, previewWidth, previewHeight);

  if (window.applyPostProcessing) {
    window.applyPostProcessing();
```

```
    }

    // Рендер закончен — чисто и надежно убираем экран загрузки
    document.getElementById('statusOverlay').classList.add('hidden');
    document.getElementById('loaderSpinner').style.display = 'none';

    if (window.toggleUIBlock) window.toggleUIBlock(false);
    document.getElementById('secEffects').classList.remove('disabled-control');
    if (window.toggleFiltersState) window.toggleFiltersState(false);
    document.getElementById('downloadBtn').disabled = false;
    document.getElementById('resetBtn').disabled = false;

    // Блокируем кнопку до выбора новых файлов
    document.getElementById('processBtn').disabled = true;
};

// Склеивание оригинальных MP4 по байтам (Blob Concat)
document.getElementById('downloadVideoBtn').addEventListener('click', () => {
    if (!window.astroState.videoFiles || window.astroState.videoFiles.length < 2) return;

    const videoBlob = new Blob(window.astroState.videoFiles, { type: 'video/mp4' });
    const url = URL.createObjectURL(videoBlob);

    const a = document.createElement('a');
    a.href = url;
    a.download = `Combined_AstroTimelapse_${Date.now()}.mp4`;
    a.click();

    setTimeout(() => URL.revokeObjectURL(url), 100);
});
```

=== ФАЙЛ: script.js ===

```
window.astroState = {
    videoFiles: [], // Список пачки видео
    cachedFrames: [], // Кэш кадров в ОЗУ
    rawCompositeBuffer: null,
    previewBuffer: null
};

const videoInput = document.getElementById('videoInput');
const uploadBtnClick = document.getElementById('uploadBtnClick');
const fileInfoText = document.getElementById('fileInfoText');
const dropZone = document.getElementById('dropZone');
const startOpacityInput = document.getElementById('startOpacity');
const opacityVal = document.getElementById('opacityVal');
const processBtn = document.getElementById('processBtn');
const downloadBtn = document.getElementById('downloadBtn');
const downloadVideoBtn = document.getElementById('downloadVideoBtn');
const resetBtn = document.getElementById('resetBtn');
const tutorialBlock = document.getElementById('tutorialBlock');
const closeTutorialBtn = document.getElementById('closeTutorialBtn');

const canvas = document.getElementById('glCanvas');
const ctx = canvas.getContext('2d');

if (closeTutorialBtn && tutorialBlock) {
    closeTutorialBtn.addEventListener('click', () => {
        tutorialBlock.style.display = 'none';
    });
}

window.enhanceSlider = function(input) {
    const wrapper = document.createElement('div');
    wrapper.className = 'slider-controls';
    input.parentNode.insertBefore(wrapper, input);

    const btnMinus = document.createElement('button');
    btnMinus.className = 'btn-step'; btnMinus.innerHTML = '-'; btnMinus.type = 'button';
    const btnPlus = document.createElement('button');
    btnPlus.className = 'btn-step'; btnPlus.innerHTML = '+'; btnPlus.type = 'button';

    wrapper.appendChild(btnMinus); wrapper.appendChild(input); wrapper.appendChild(btnPlus);
```

```
    const changeVal = (delta) => {
        if (input.disabled) return;
        let val = parseFloat(input.value) + delta;
        if (val < parseFloat(input.min)) val = parseFloat(input.min);
        if (val > parseFloat(input.max)) val = parseFloat(input.max);
        input.value = val;
        input.dispatchEvent(new Event('input'));
    };
    btnMinus.addEventListener('click', () => changeVal(-1));
    btnPlus.addEventListener('click', () => changeVal(1));
    wrapper.addEventListener('wheel', (e) => {
        if (input.disabled) return;
        e.preventDefault();
        changeVal(e.deltaY < 0 ? 1 : -1);
    }, { passive: false });
};

window.enhanceSlider(startOpacityInput);

startOpacityInput.addEventListener('input', (e) => {
    opacityVal.textContent = e.target.value + '%';
    if(window.astroState.cachedFrames.length > 0 && window.renderCompositeTracks) {
        window.renderCompositeTracks();
    }
});

resetBtn.addEventListener('click', () => {
    startOpacityInput.value = 20;
    opacityVal.textContent = '20%';
    if (window.resetAllEffects) window.resetAllEffects();
    if (window.renderCompositeTracks) window.renderCompositeTracks();
});

['dragenter', 'dragover'].forEach(name => {
    dropZone.addEventListener(name, (e) => { e.preventDefault(); dropZone.classList.add('dragover'); });
});
['dragleave', 'drop'].forEach(name => {
    dropZone.addEventListener(name, (e) => { e.preventDefault(); dropZone.classList.remove('dragover'); });
});
dropZone.addEventListener('drop', (e) => {
    e.preventDefault();
    if(e.dataTransfer.files.length) handleFiles(e.dataTransfer.files);
});

uploadBtnClick.addEventListener('click', () => videoInput.click());
videoInput.addEventListener('change', (e) => {
    if(e.target.files.length) handleFiles(e.target.files);
});

function handleFiles(fileList) {
    if (!fileList || fileList.length === 0) return;

    const filesArray = Array.from(fileList).filter(f =>
        f.type.startsWith('video/') || f.name.toLowerCase().endsWith('.mp4') || f.name.toLowerCase().endsWith('.mov')
    );

    if (filesArray.length === 0) {
        document.getElementById('statusIcon').className = "fa-solid fa-circle-exclamation overlay-icon";
        document.getElementById('statusTitle').textContent = "Формат не поддерживается";
        document.getElementById('statusText').textContent = "Пожалуйста, выберите корректные файлы видео (MP4, MOV, AVI).";
        return;
    }

    filesArray.sort((a, b) => a.name.localeCompare(b.name, undefined, {numeric: true, sensitivity: 'base'}));

    // КРИТИЧЕСКИЙ ФИКС ОЧИСТКИ КЭША: Начисто стираем старые кадры из ОЗУ ноутбука перед загрузкой новых
    if(window.astroState.cachedFrames.length) {
        window.astroState.cachedFrames.forEach(b => { if(b && b.close) b.close(); });
    }
    window.astroState.cachedFrames = [];
    window.astroState.rawCompositeBuffer = null;
    window.astroState.previewBuffer = null;

    window.astroState.videoFiles = filesArray;
```

```
    fileInfoText.textContent = `Загружено видео: ${filesArray.length} шт.`;

    secTracks.classList.remove('disabled-control');
    startOpacityInput.disabled = false;
    processBtn.disabled = false; // Разблокируем кнопку, так как пришли новые файлы

    if (filesArray.length >= 2) {
        downloadVideoBtn.style.display = 'inline-flex';
        downloadVideoBtn.disabled = false;
    } else {
        downloadVideoBtn.style.display = 'none';
        downloadVideoBtn.disabled = true;
    }

    document.getElementById('statusIcon').className = "fa-solid fa-circle-play overlay-icon";
    document.getElementById('statusTitle').textContent = "Пакет видео загружен";
    document.getElementById('statusText').textContent = `Сортировка выполнена успешно. Всего видеоклипов: ${filesArray.length}.
Нажмите кнопку «Сгенерировать треки».`;
}

window.toggleUIBlock = function(isBlocked) {
    processBtn.disabled = isBlocked;
    uploadBtnClick.disabled = isBlocked;
    startOpacityInput.disabled = isBlocked;
    downloadVideoBtn.disabled = isBlocked;
    dropZone.style.pointerEvents = isBlocked ? 'none' : 'auto';
    startOpacityInput.parentNode.querySelectorAll('.btn-step').forEach(b => b.disabled = isBlocked);

    if(isBlocked) {
        resetBtn.disabled = true;
    } else {
        document.getElementById('progressContainer').style.display = 'none';
    }
};
```

=== ФАЙЛ: style.css ===

```
:root {
    --bg-main: #0c0d10;
    --bg-sidebar: #13151a;
    --bg-card: #1c1e24;
    --bg-card-hover: #242730;
    --accent: #3b82f6;
    --accent-dim: rgba(59, 130, 246, 0.12);
    --text-main: #f3f4f6;
    --text-muted: #9ca3af;
    --border-color: #262930;
    --radius-lg: 12px;
    --radius-md: 8px;
}

* { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Inter', sans-serif; -webkit-font-smoothing: antialiased; }
body { background-color: var(--bg-main); color: var(--text-main); overflow: hidden; height: 100vh; width: 100vw; }

.editor-layout { display: grid; grid-template-columns: 340px 1fr; height: 100vh; width: 100vw; }
.sidebar { background-color: var(--bg-sidebar); border-right: 1px solid var(--border-color); display: flex; flex-direction: column; height:
100vh; order: 1; }

.sidebar-header { padding: 16px 20px; border-bottom: 1px solid var(--border-color); display: flex; justify-content: center; align-items:
center; gap: 16px; }
.app-logo-img { width: 32px; height: 32px; object-fit: contain; border-radius: var(--radius-md);}
.logo-icon { font-size: 24px; color: var(--accent); }
.sidebar-header h2 { font-size: 32px; font-weight: 400; letter-spacing: -0.3px; }
.version { font-size: 10px; color: var(--text-muted); background: #262930; padding: 2px 6px; border-radius: 4px; display: inline-block;
margin-top: 4px; }

/* Виджет туториала */
.tutorial-card {
    background: linear-gradient(135deg, #1d2433 0%, #151a24 100%);
    border: 1px solid #2b384e;
    border-radius: var(--radius-lg);
    padding: 14px;
    margin-bottom: 4px;
}
```

```
.tutorial-header { display: flex; justify-content: space-between; align-items: center; font-size: 12px; font-weight: 700; color: #60a5fa; }
.tutorial-header button { background: none; border: none; color: var(--text-muted); cursor: pointer; font-size: 14px; }
.tutorial-header button:hover { color: #fff; }
.tutorial-steps { font-size: 11px; color: #93a3b8; margin-top: 8px; padding-left: 16px; line-height: 1.5; }
.tutorial-steps li { margin-bottom: 6px; }

#fileInfoText { font-size: 11px; color: var(--text-muted); margin-top: 8px; text-align: center; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.sidebar-content { flex: 1; overflow-y: auto; padding: 20px; display: flex; flex-direction: column; gap: 16px; }
.section-card { background-color: var(--bg-card); border: 1px solid var(--border-color); border-radius: var(--radius-lg); padding: 16px; transition: opacity 0.2s ease; }
.section-card.disabled-control { opacity: 0.25; pointer-events: none; }
.section-title { font-size: 11px; font-weight: 700; text-transform: uppercase; letter-spacing: 0.5px; color: var(--text-muted); margin-bottom: 14px; }

.slider-group { margin-bottom: 14px; }
.slider-group:last-child { margin-bottom: 0; }
.slider-header { display: flex; justify-content: space-between; font-size: 12px; margin-bottom: 8px; color: var(--text-main); }
.slider-header .val { color: var(--accent); font-weight: 600; font-variant-numeric: tabular-nums; }
.slider-controls { display: flex; align-items: center; gap: 10px; }

.btn-step {
    background: #262930; border: 1px solid var(--border-color); color: var(--text-main); width: 32px; height: 32px; border-radius: 6px;
    cursor: pointer; display: flex; align-items: center; justify-content: center; font-size: 14px; font-weight: 700; transition: all 0.15s ease;
}
.btn-step:hover:not(:disabled) { background: #3b82f6; border-color: #3b82f6; color: #fff; }
.btn-step:disabled { opacity: 0.2; cursor: not-allowed; }

input[type="range"] { flex: 1; -webkit-appearance: none; height: 8px; background: #2d3139; border-radius: 4px; outline: none; cursor: pointer; }
input[type="range"]::-webkit-slider-thumb {
    -webkit-appearance: none; width: 18px; height: 18px; border-radius: 50%; background: #ffffff; cursor: pointer; border: 3px solid var(--bg-card);
    box-shadow: 0 2px 4px rgba(0,0,0,0.3); transition: transform 0.1s, background-color 0.1s;
}
input[type="range"]::-webkit-slider-thumb:hover { transform: scale(1.15); background: var(--accent); }

.rgb-slider.r-slider .val { color: #f87171; }
.rgb-slider.g-slider .val { color: #4ade80; }
.rgb-slider.b-slider .val { color: #60a5fa; }
.divider { height: 1px; background: var(--border-color); margin: 16px 0; }

.btn { display: inline-flex; align-items: center; justify-content: center; gap: 8px; height: 42px; padding: 0 16px; border-radius: var(--radius-md); font-size: 13px; font-weight: 600; border: none; cursor: pointer; transition: all 0.2s ease; width: 100%; }
.btn-primary { background-color: var(--accent); color: white; }
.btn-primary:hover:not(:disabled) { background-color: #2563eb; }
.btn-primary:disabled { background-color: #262930; color: var(--text-muted); cursor: not-allowed; }
.sidebar-footer { padding: 16px 20px; border-top: 1px solid var(--border-color); display: flex; flex-direction: column; background: var(--bg-sidebar); }
.btn-tonal { background-color: #262930; color: var(--text-main); border: 1px solid var(--border-color); }
.btn-tonal:hover:not(:disabled) { background-color: #2d3139; border-color: #4b5563; }
.btn-tonal:disabled { opacity: 0.2; cursor: not-allowed; }

.viewport { background-color: var(--bg-main); display: flex; justify-content: center; align-items: center; padding: 24px; height: 100vh; position: relative; transition: background-color 0.2s ease; order: 2; }
.viewport.dragover { background-color: #111622; }
.viewport.dragover .canvas-container { border: 2px dashed var(--accent); }
.canvas-container { max-width: 100%; max-height: 100%; position: relative; border-radius: var(--radius-lg); box-shadow: 0 20px 40px rgba(0,0,0,0.5); overflow: hidden; background-color: #050507; display: flex; justify-content: center; align-items: center; border: 2px solid transparent; transition: border 0.2s ease; }
canvas { max-width: 100%; max-height: 85vh; object-fit: contain; display: block; background: #000; }

.viewport-overlay { position: absolute; top: 0; left: 0; width: 100%; height: 100%; background: #111317; display: flex; flex-direction: column; justify-content: center; align-items: center; padding: 32px; text-align: center; z-index: 100; }
.viewport-overlay.hidden { display: none !important; }
.overlay-icon { font-size: 48px; color: var(--text-muted); margin-bottom: 16px; }
.status-title { font-size: 16px; font-weight: 600; margin-bottom: 8px; }
.status-desc { font-size: 13px; color: var(--text-muted); max-width: 360px; line-height: 1.5; }
.progress-container { width: 260px; height: 4px; background: #262930; border-radius: 2px; margin-top: 20px; overflow: hidden; display: none; }
.progress-bar { height: 100%; width: 0%; background: var(--accent); transition: width 0.05s linear; }
.loader-spinner { width: 40px; height: 40px; border: 3px solid #262930; border-top-color: var(--accent); border-radius: 50%; animation: spin 0.8s linear infinite; margin-bottom: 16px; display: none; }
@keyframes spin { to { transform: rotate(360deg); } }
```

```
@media (max-width: 768px) {
    body { overflow-y: auto; }
    .editor-layout { grid-template-columns: 1fr; grid-template-rows: auto 1fr; height: auto; overflow-y: auto; }
    .viewport { height: 50vh; padding: 12px; border-bottom: 1px solid var(--border-color); position: sticky; top: 0; z-index: 150;
background: var(--bg-main); }
    canvas { max-height: 45vh; }
    .sidebar { height: auto; border-right: none; border-top: 1px solid var(--border-color); }
    .sidebar-header { padding: 12px 16px; }
    .app-logo-img { width: 44px; height: 44px; }
    .sidebar-content { padding: 12px; overflow-y: visible; }
    .sidebar-footer { position: sticky; bottom: 0; z-index: 200; box-shadow: 0 -8px 24px rgba(0,0,0,0.5); }
}

/* Кастомизация скроллбара для левой панели (Webkit браузеры: Chrome, Яндекс, Opera, Edge) */
.sidebar-content::-webkit-scrollbar {
    width: 6px; /* Делаем полосу очень тонкой */
    height: 6px;
}

.sidebar-content::-webkit-scrollbar-track {
    background: transparent; /* Полностью прозрачный фон дорожки */
}

.sidebar-content::-webkit-scrollbar-thumb {
    background-color: #2d3139; /* Цвет бегунка в тон границам карточек */
    border-radius: 20px; /* Идеальное скругление */
    transition: background-color 0.2s ease;
}

.sidebar-content::-webkit-scrollbar-thumb:hover {
    background-color: var(--accent); /* При наведении бегунок плавно подсвечивается синим */
}

/* Поддержка Firefox */
.sidebar-content {
    scrollbar-width: thin;
    scrollbar-color: #2d3139 transparent;
}

/* Красивый круговой лоадер в стиле Google Material Design */
.google-spinner {
    width: 50px;
    height: 50px;
    display: none; /* По умолчанию скрыт */
    margin-bottom: 24px;
    position: relative;
}

.google-spinner::before {
    content: '';
    position: absolute;
    top: 0; left: 0; width: 100%; height: 100%;
    border-radius: 50%;
    border: 4px solid var(--accent-dim); /* Тусклая подложка круга */
}

.google-spinner::after {
    content: '';
    position: absolute;
    top: 0; left: 0; width: 100%; height: 100%;
    border-radius: 50%;
    border: 4px solid transparent;
    border-top-color: var(--accent); /* Яркий бегунок */
    animation: google-spin 0.8s cubic-bezier(0.4, 0, 0.2, 1) infinite;
}

@keyframes google-spin {
    0% { transform: rotate(0deg); }
    100% { transform: rotate(360deg); }
}

/* Скрываем старый сломанный контейнер с полоской */
#progressContainer {
```

```
    display: none !important;
}
```